

1.6 Generación de la tabla de símbolo y de direcciones

La tabla de símbolos permite al compilador relacionar los nombres utilizados en un programa con los elementos que representan. Un símbolo puede corresponder, entre otros elementos, a una variable local, un parámetro, un método, un campo o un tipo. Esta información ayuda a determinar a qué declaración se refiere cada uso de un identificador y qué reglas del lenguaje se le aplican. En el compilador de C# accesible mediante Roslyn, los símbolos y el modelo semántico permiten consultar, por ejemplo, el elemento al que se refiere un nombre y el tipo de una expresión

El **ámbito** determina dónde puede utilizarse un nombre. Por ello, identificar un símbolo requiere considerar el contexto en que aparece: dos identificadores escritos igual no necesariamente se refieren al mismo elemento. Si un nombre no existe en el contexto donde se utiliza, C# puede emitir el error **CS0103**.

Tabla de Símbolos

La tabla de símbolos es una estructura de datos utilizada por el compilador para almacenar información sobre los identificadores (variables, funciones, constantes, etc.) utilizados en el programa fuente. Cada entrada en la tabla de símbolos contiene información como el nombre del identificador, su tipo de datos, su ubicación en la memoria, su ámbito (scope), y otra información relevante. La tabla de símbolos es esencial para llevar un seguimiento de las declaraciones y usos de identificadores en el programa y garantizar que se utilicen de manera coherente y correcta.

Durante el proceso de compilación, el compilador escanea el código fuente en busca de declaraciones de identificadores y los agrega a la tabla de símbolos. Si se encuentra una declaración de variable, por ejemplo, el compilador almacenará su nombre, tipo y otra información en la tabla de símbolos. Luego, cuando se encuentren usos de ese identificador en el código, el compilador buscará en la tabla de símbolos para asegurarse de que se utilice de manera correcta y coherente.

Generación de Direcciones

La generación de direcciones es una etapa crucial del proceso de compilación en la que el compilador asigna ubicaciones de memoria específicas a las variables y otros elementos del programa. Esto implica la asignación de direcciones de memoria para cada identificador declarado en el programa, de modo que el programa pueda acceder y manipular los datos en tiempo de ejecución.

El proceso de generación de direcciones implica varias tareas, como:

1. **Reserva de memoria:** El compilador debe asignar suficiente espacio de

memoria para cada variable en función de su tipo de datos. Por ejemplo, las variables enteras suelen requerir 4 bytes de memoria en sistemas de 32 bits.

2. **Asignación de direcciones:** El compilador asigna direcciones de memoria únicas a cada variable y luego actualiza la tabla de símbolos con estas direcciones.
3. **Generación de código de acceso:** El compilador genera código de máquina o código intermedio que utiliza las direcciones asignadas para acceder a las variables y realizar operaciones en ellas.
4. **Manejo de ámbito:** El compilador debe tener en cuenta el ámbito de las variables para garantizar que se utilicen las direcciones correctas en cada contexto.

Tabla de Símbolos

1. **Información Almacenada:** En la tabla de símbolos, se almacena información relevante para cada identificador en el programa. Esto incluye el nombre del identificador, su tipo de datos (entero, flotante, cadena, etc.), su alcance (scope), su dirección de memoria (en el caso de variables), y otros atributos como valores iniciales o tamaños en el caso de arreglos.
2. **Ámbito (Scope):** El ámbito se refiere al alcance en el cual un identificador es válido. Por ejemplo, en muchos lenguajes de programación, una variable declarada dentro de una función solo es válida dentro de esa función (ámbito local), mientras que una variable declarada fuera de todas las funciones es válida en todo el programa (ámbito global). La tabla de símbolos ayuda a rastrear estos ámbitos y a garantizar que los identificadores se utilicen correctamente dentro de sus ámbitos.
3. **Resolución de Nombres:** Cuando se encuentra un identificador en el código fuente, el compilador busca su entrada correspondiente en la tabla de símbolos para obtener información relevante, como su tipo o dirección. Esto es esencial para verificar que las operaciones que se realizan con el identificador sean coherentes con su tipo y alcance.
4. **Detectar Errores:** La tabla de símbolos también se utiliza para detectar errores en el código fuente, como el uso de un identificador no declarado o la redefinición de un identificador en el mismo ámbito.

Ejemplo

Este ejemplo en C# ilustra cómo se generarían la tabla de símbolos y las direcciones en un programa más completo. La generación de código de acceso involucra el uso de la pila y las direcciones de memoria específicas para las variables locales y globales.

```
using System;

class Program
{
    static int globalVariable = 100; // Variable global

    static void Main()
    {
        int localVar1 = 5; // Variable local en Main()
        int localVar2 = 10; // Otra variable local en Main()

        CalculateSum(localVar1, localVar2); // Llamada a una función
    }

    static void CalculateSum(int a, int b)
    {
        int result = a + b; // Variable local en CalculateSum()

        Console.WriteLine("La suma es: " + result);
    }
}
```

Generación de la Tabla de Símbolos:

1. El compilador analiza el código fuente de C# y crea una tabla de símbolos que incluye:

- ``globalVariable``: Una variable global de tipo entero.
- ``Main``: La función principal del programa.
- ``localVar1``: Una variable local en la función ``Main``.
- ``localVar2``: Otra variable local en la función ``Main``.
- ``CalculateSum``: Una función que toma dos parámetros de tipo entero.

Generación de Direcciones:

1. El compilador asigna direcciones de memoria para las variables globales y locales:

- ``globalVariable`` podría asignarse a una dirección específica en la memoria de datos globales.
- ``localVar1`` podría asignarse a una dirección en la pila de la función ``Main``.
- ``localVar2`` también podría asignarse a una dirección en la pila de la función ``Main``.

2. La función ``CalculateSum`` toma dos parámetros ``a`` y ``b``, que también se almacenan en la pila de la función cuando se llama.

Generación de Código de Acceso:

1. Cuando se asignan valores a ``localVar1`` y ``localVar2`` en la función ``Main``, el compilador generará código para almacenar los valores en las direcciones de memoria correspondientes.
2. Cuando se llama a la función ``CalculateSum``, el compilador generará código para pasar los valores de ``localVar1`` y ``localVar2`` a los parámetros ``a`` y ``b`` de la función.
3. Dentro de ``CalculateSum``, el compilador generará código para realizar la suma y mostrar el resultado.

1.7 Manejo de errores semánticos

¿Qué son los errores semánticos?

Los errores semánticos son problemas que se originan en el mal uso o comprensión del significado de las construcciones del lenguaje en un programa. Estos errores pueden ser difíciles de detectar durante la fase de compilación y generalmente solo se revelan durante la ejecución del programa o cuando se realizan pruebas.

Categorías de Errores Semánticos

Errores de Inicialización: Uso de variables sin inicializar.

Errores de Tipo de Datos: Tratar una cadena como un número, o viceversa.

Errores de Lógica: Usar el operador equivocado en una comparación, condiciones mal definidas en loops.

Errores de Alcance: Uso incorrecto de variables globales y locales.

Errores de Recurso: Como desbordamientos de memoria y fugas.

Estrategias para el Manejo de Errores Semánticos

Validación de Entrada: Comprobar si los datos de entrada son válidos antes de procesarlos.

Asertos: Usar declaraciones de asertos para comprobar la validez de los estados del programa.

Pruebas Unitarias y de Integración: Verificar que cada unidad de código y conjuntos de unidades funcionan como se espera.

Patrones de Diseño para el Manejo de Errores

Patrón Singleton para Registro de Errores: Utilizar un único objeto a lo largo del proyecto para manejar el registro de errores.

Factory Method para la Creación de Excepciones: Utilizar un método de fábrica para crear excepciones personalizadas para manejar errores específicos.

Chain of Responsibility para el Escalado de Errores: Utilizar una cadena de objetos para manejar errores de forma jerárquica.

Tratamiento y Recuperación de Errores

Abortar y Reiniciar: Terminar la operación fallida y, opcionalmente, reiniciarla.

Reintentar: Intentar la operación nuevamente, posiblemente después de una pequeña pausa o después de cambiar ciertas condiciones.

Reportar y Continuar: Registrar el error y seguir con la siguiente tarea si es posible.

Salir con Gracia: Si un error es irreparable, terminar el programa de manera controlada, liberando todos los recursos y notificando al usuario de la mejor manera posible.

Evaluación y Monitorización

Sistemas de Monitorización de Errores en Tiempo Real: Herramientas como Sentry, Loggly, etc., para detectar y notificar errores en tiempo real.

Auditorías de Código y Revisiones por Pares: Evaluar regularmente el código para detectar patrones de errores semánticos y corregirlos.

El manejo de errores semánticos es un área crítica que requiere una combinación de prácticas de codificación robustas, estrategias de diseño de software y herramientas de monitorización para manejar de forma efectiva.